CODICODEC: UNIFYING CONTINUOUS AND DISCRETE COMPRESSED REPRESENTATIONS OF AUDIO

Marco Pasini[1] **Stefan Lattner**[2] **György Fazekas**[1]
[1]Queen Mary University of London, UK [2]Sony Computer Science Laboratories, Paris, France
m.pasini@qmul.ac.uk

ABSTRACT

Efficiently representing audio signals in a compressed latent space is critical for latent generative modelling. However, existing autoencoders often force a choice between continuous embeddings and discrete tokens. Furthermore, achieving high compression ratios while maintaining audio fidelity remains a challenge. We introduce CoDiCodec, a novel audio autoencoder that overcomes these limitations by both efficiently encoding global features via *summary embeddings*, and by producing *both* compressed continuous embeddings at ~11 Hz *and* discrete tokens at a rate of 2.38 kbps from the *same* trained model, offering unprecedented flexibility for different downstream generative tasks. This is achieved through Finite Scalar Quantization (FSQ) and a novel *FSQ-dropout* technique, and does not require additional loss terms beyond the single consistency loss used for end-to-end training. CoDiCodec supports both autoregressive decoding and a novel *parallel* decoding strategy, with the latter achieving superior audio quality and faster decoding. CoDiCodec outperforms existing continuous and discrete autoencoders at similar bitrates in terms of reconstruction audio quality. Our work enables a unified approach to audio compression, bridging the gap between continuous and discrete generative modelling paradigms.

1. INTRODUCTION

Efficient, compact audio representations are crucial for applications in Music Information Retrieval (MIR), generative modelling, and compression. While recent advances in deep learning have demonstrated impressive results in the learning of compressed representations, several key challenges remain. These include balancing high compression ratios with reconstruction fidelity, enabling both discrete and continuous latent representations for diverse downstream applications, and achieving efficient training and inference without resorting to a complex and unstable training process.

Existing audio autoencoders often fall short in one or more of these areas. Vector Quantization (VQ)-based approaches, such as SoundStream [1], EnCodec [2], and Descript Audio Codec (DAC, [3]), can excel at high-fidelity reconstruction and are well-suited for training autoregressive language models on the resulting discrete latent tokens [4–6]. However, their discrete nature makes them less compatible with continuous generative frameworks (e.g., GANs [7], diffusion models [8–10]), as their pre-quantization continuous features are typically high-dimensional and unsuitable for efficient latent modelling. Continuous autoencoders, such as those used in Moûsai [11], in Musika [12, 13], and in the Stable Audio family of generative models [14–16], address the compatibility issue with continuous latent generative models. However, they often require multi-stage training procedures, unstable adversarial training objectives, or slow iterative decoding processes. While Music2Latent [17] introduces a consistency-based autoencoder that achieves single-step decoding and single-loss end-to-end training, it is limited to continuous representations. Furthermore, most continuous autoencoders encode audio into temporally ordered sequences, leading to redundancy by repeatedly encoding global features across embeddings.

This paper introduces CoDiCodec (Continuous-Discrete Codec), a novel audio autoencoder that addresses these limitations. CoDiCodec achieves the following key objectives:

- Encoding of both **compressed continuous embeddings** (~11 Hz) and **discrete tokens** (2.38 kbps) of 44.1 kHz stereo audio from a single model, offering flexibility for downstream tasks without the need for separate models.
- Use of **summary embeddings** [18] to capture global features, reducing redundancy compared to ordered sequences for better fidelity at similar compression
- Leveraging consistency models [19, 20], CoDiCodec is **trained end-to-end using a single loss**, simplifying the training process and avoiding the complexities of adversarial training or multi-stage procedures.
- Support for both autoregressive and a novel, faster **parallel decoding strategy** for long sequences.
- Introduction of **FSQ-dropout**, enabling higher-quality continuous decoding by bypassing quantization, while promoting informative embeddings suitable for downstream modeling.
- An **improved architecture** designed to increase the proportion of parameters used by the transformer layers compared to convolutional ones, which simplifies the process of scaling, while achieving faster inference speed compared to Music2Latent2.

To our knowledge, this is the first work unifying summary embeddings, consistency-based training, and the gen-

© M. Pasini, S. Lattner, and G. Fazekas. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** M. Pasini, S. Lattner, and G. Fazekas, "CoDiCodec: Unifying Continuous and Discrete Compressed Representations of Audio", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

eration of both continuous and discrete representations from a single audio autoencoder. Our experiments show that CoDiCodec outperforms existing continuous and discrete autoencoders in terms of reconstruction quality measured by FAD [21] with different backbones. We present comprehensive ablation studies validating the design choices.

2. RELATED WORK

2.1 Audio Autoencoders

Audio autoencoders aim to learn compressed latent representations of audio signals, typically for dimensionality reduction, generative modeling, or MIR tasks. These can be broadly divided into those producing discrete and continuous compressed latent representations.

Discrete Latent Representations: Vector Quantization (VQ [22, 23]) has been a dominant technique for learning discrete audio representations. SoundStream [1], EnCodec [2], and Descript Audio Codec (DAC) [3] use Residual Vector Quantization (RVQ) to achieve high-fidelity audio reconstruction. These models are particularly well-suited for training autoregressive language models on the resulting discrete tokens [4–6]. However, their discrete nature limits compatibility with continuous generative frameworks, and they often yield lower temporal compression, resulting in longer sequences for downstream tasks compared to continuous methods.

Continuous Latent Representations: Several approaches learn continuous latent representations of audio. The autoencoder used in Musika [12] reconstructs both magnitude and phase components of a spectrogram, enabling fast inference. However, it relies on a two-stage training process and an adversarial objective. Moûsai [11] uses a diffusion autoencoder, achieving end-to-end training but requiring expensive iterative sampling for decoding. Stable Audio and Stable Audio 2 [14–16] leverage continuous representations to train diffusion-based audio generation models, but the proposed autoencoders still require an objective with multiple adversarial and reconstruction losses. Music2Latent [17] introduces a consistency-based autoencoder, achieving single-step decoding and end-to-end training with a single loss function. However, it is limited to producing ordered sequences of continuous representations. Music2Latent2 [18] introduces summary embeddings [24] that are able to more efficiently encode global features from the input samples, while still encoding to continuous-valued latents. CoDiCodec, in contrast, can encode both continuous and discrete representations, while still using summary embeddings.

2.2 Consistency Models

Consistency models [19, 20] represent a class of generative models that enables fast one-step generation. While showing impressive results in image generation [25], their application to audio remains under-explored. CoMoSpeech [26] explores consistency distillation for speech synthesis, requiring a pre-trained teacher. Music2Latent [17] and Music2Latent2 [18] were the first to use consistency models in an end-to-end audio autoencoder framework.

3. BACKGROUND

3.1 Consistency Models

Consistency models [19,20] are a class of generative models that learn to map any point on a diffusion process trajectory back to the origin of that trajectory. They are based on the probability flow (PF) ordinary differential equation (ODE) [27], which describes the evolution of a data sample x perturbed by Gaussian noise with standard deviation σ:

$$\frac{dx}{d\sigma} = -\sigma \nabla_x \log p_\sigma(x), \quad \sigma \in [\sigma_{\text{min}}, \sigma_{\text{max}}]. \tag{1}$$

where $p_\sigma(x)$ is the perturbed data distribution, and $\nabla_x \log p_\sigma(x)$ is the score function. The PF ODE defines trajectories mapping noisy samples x_σ to the clean sample $x_{\sigma_{\text{min}}}$ (where $\sigma_{\text{min}} \approx 0$). Consistency models learn a consistency function $f(x_\sigma, \sigma)$ that directly maps any point on this trajectory to its origin: $f(x_\sigma, \sigma) \mapsto x_{\sigma_{\text{min}}}$, while satisfying the boundary condition $f(x_{\sigma_{\text{min}}}, \sigma_{\text{min}}) = x_{\sigma_{\text{min}}}$. A consistency model $f_\theta(x_\sigma, \sigma)$ is a neural network parameterized by θ that approximates the true consistency function. To enforce the boundary condition, consistency models are typically parameterized as $f_\theta(x_\sigma, \sigma) = c_{\text{skip}}(\sigma)x_\sigma + c_{\text{out}}(\sigma)F_\theta(x_\sigma, \sigma)$, where $F_\theta(x_\sigma, \sigma)$ is a neural network, and $c_{\text{skip}}(\sigma)$ and $c_{\text{out}}(\sigma)$ are chosen such that $c_{\text{skip}}(\sigma_{\text{min}}) = 1$ and $c_{\text{out}}(\sigma_{\text{min}}) = 0$ to satisfy the boundary condition.

3.2 Consistency Training

Consistency models can be trained via Consistency Distillation (CD), requiring a pre-trained diffusion model, or Consistency Training (CT). CoDiCodec uses CT, allowing for training in isolation without a pretrained teacher model. In CT, the continuous PF ODE (Eq. 1) is discretized using a sequence of noise levels $\sigma_{\text{min}} = \sigma_1 < \sigma_2 < \cdots < \sigma_N = \sigma_{\text{max}}$. The consistency model is trained by minimizing the following loss:

$$\mathcal{L}_{\text{CT}} = \mathbb{E}\left[\lambda(\sigma_i, \sigma_{i+1}) d\left(f_\theta(x_{\sigma_{i+1}}, \sigma_{i+1}), f_{\theta^-}(x_{\sigma_i}, \sigma_i)\right)\right], \tag{2}$$

where $x \sim p_{\text{data}}$ is a training sample, σ_i and σ_{i+1} are adjacent noise levels, x_{σ_i} and $x_{\sigma_{i+1}}$ are corresponding noisy versions of x, $d(x, y)$ is a distance metric, and $\lambda(\sigma_i, \sigma_{i+1})$ is a weighting function. f_θ is the student model, and f_{θ^-} is the teacher, with parameters $\theta^- \leftarrow \text{stopgrad}(\theta)$. The loss minimizes the distance between model outputs at adjacent noise steps σ_i, σ_{i+1}, using the teacher f_{θ^-} to provide the targets. Post-training, generation from noise $x_{\sigma_{\text{max}}}$ can occur in one step ($x = f_\theta(x_{\sigma_{\text{max}}}, \sigma_{\text{max}})$) where $x_{\sigma_{\text{max}}} \sim \mathcal{N}(0, \sigma_{\text{max}}^2 I)$, or multiple steps.

3.3 Finite Scalar Quantization (FSQ)

Finite Scalar Quantization (FSQ) [28] is a simple quantization technique and, unlike Vector Quantization (VQ [22, 23]), it does not require additional loss terms. It is also shown to achieve almost perfect codebook utilization

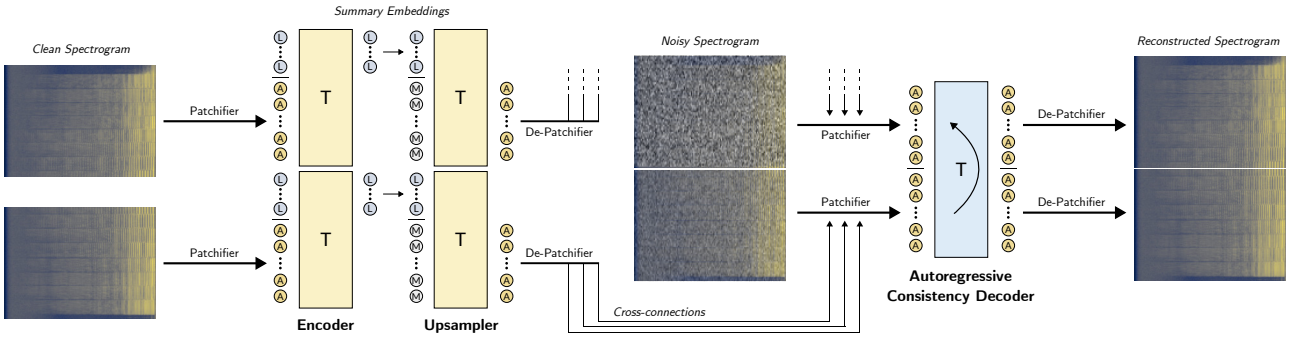


Figure 1. Training process. Transformer modules are represented with *T*, audio embeddings with *A*, learned/summary embeddings with *L*, and mask embeddings with *M*. We represent chunked causal masking with a curved arrow.

even with large codebook sizes. FSQ bounds a value x in $[-N, N]$ where N is integer, rounds it, and rescales:

$$\hat{x} = \frac{\text{round}(N \cdot \tanh(x))}{N}, \tag{3}$$

where $\hat{x}$ is the quantized value, and round$(\cdot)$ denotes the rounding operation. Applied element-wise to a D-dimensional vector $\mathbf{x}$, each element $\hat{x}_i$ takes one of $2N+1$ discrete values in $[-1, 1]$, yielding an implicit codebook of size $(2N+1)^D$. The gradient of the non-differentiable rounding operation is approximated using the straight-through estimator [29].

4. CODICODEC

Following previous work [17, 18, 30, 31], CoDiCodec operates on complex Short-Time Fourier Transform (STFT) spectrograms. To address the skewed distribution of different frequency bins, we apply an amplitude transformation [32]: $\tilde{c} = \beta|c|^{\alpha}e^{i\angle(c)}$, where c and $\tilde{c}$ are the original and transformed STFT coefficients, $\alpha \in (0, 1]$ is a compression exponent that emphasizes lower-energy components, $\angle(c)$ is the phase angle of c, and $\beta \in \mathbb{R}^+$ is a scaling factor. We treat the complex spectrogram as a two-channel (real/imaginary) representation.

4.1 Architecture

The proposed architecture (Fig. 1) consists of an encoder, an upsampler, and a consistency model decoder. The model operates on pairs of consecutive audio chunks.

Encoder: It takes a spectrogram chunk $x \in \mathbb{R}^{C \times F \times T}$ ($C = 2 \times$ channels, F and T are the number of frequency bins and time frames) and downsamples it via a convolutional patchifier. The flattened features (audio embeddings) are concatenated with K learnable *summary embeddings* and fed into transformer blocks [33] (T in Fig. 1) for summary embeddings to gather global context. Only the K summary embeddings are retained, projected to d_{lat}, and processed via $\tanh$ (for continuous output) or FSQ (for discrete tokens converted to indices at inference).

Upsampler: It mirrors the encoder structure but upsamples instead of downsampling. It takes K summary embeddings (discrete tokens are mapped back to vectors), concatenates learnable mask embeddings, and processes them through transformer blocks to "de-compress" information from the summary embeddings. The resulting audio embeddings are reshaped and upsampled by a convolutional de-patchifier. Its sole purpose is providing intermediate feature maps as *cross-connections* to the decoder: since the consistency model decoder generates samples in one step, it is crucial to provide information about which sample to reconstruct to the first layers of the decoder [17].

Consistency Decoder: It is trained to map a noisy spectrogram x_σ to a clean one, conditioned on upsampler cross-connections. A patchifier downsamples the input noisy spectrogram x_σ. Cross-connections from the upsampler are added to feature maps at each resolution level: this is possible because of the exact symmetry of the patchifier with respect to the de-patchifier of the upsampler. The output is flattened and fed into a stack of transformer blocks. Crucially, transformers operate on consecutive chunk pairs $(x_{\sigma,\text{left}}, x_{\sigma,\text{right}})$ with chunked causal masking (right chunk attends to left, not vice-versa) to enable autoregressive decoding. A de-patchifier upsamples the output to the original spectrogram dimension. Skip connections additively combine the feature maps from the patchifier to the corresponding ones in the de-patchifier. The forward pass is:

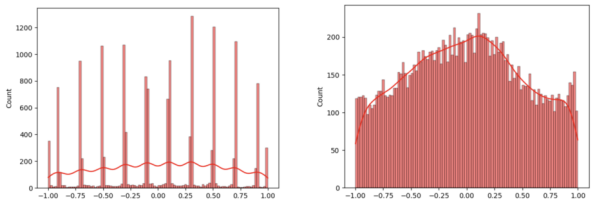
$$\hat{x}_{\text{left}}, \hat{x}_{\text{right}} = \text{Dec}_{\sigma_{\text{left}},\sigma_{\text{right}}}(\text{Up}(\text{Enc}(x_{\text{left}})), x_{\text{left}} + \sigma_{\text{left}}\varepsilon_{\text{left}}, \text{Up}(\text{Enc}(x_{\text{right}})), x_{\text{right}} + \sigma_{\text{right}}\varepsilon_{\text{right}})$$

where Enc, Up, and Dec are the Encoder, Upsampler, and Decoder. $\varepsilon \sim \mathcal{N}(0, I)$ and noise levels σ are sampled independently. End-to-end training uses the consistency loss [20]:

$$\mathcal{L} = \mathbb{E}\left[\frac{1}{\Delta\sigma} d\left(\text{Dec}_{\sigma_{\text{left}}+\Delta\sigma,\sigma_{\text{right}}+\Delta\sigma}, \text{sg}\left(\text{Dec}_{\sigma_{\text{left}},\sigma_{\text{right}}}\right)\right)\right]$$

with Pseudo-Huber distance $d(\cdot)$, step $\Delta\sigma$, and stop-gradient *sg*. We use the EDM parameterization [19, 34], continuous log-normal noise sampling [34], and an exponential $\Delta\sigma$ schedule [17, 35].

FSQ-dropout: To enable decoding from both discrete FSQ tokens and more expressive continuous embeddings using the same model, we introduce *FSQ-dropout*. Standard FSQ training causes continuous pre-quantization values ($\tanh(\mathbf{z})$) to cluster near quantization levels (Fig. 2(a)),



(a) Standard FSQ (b) FSQ-dropout p=0.75

Figure 2. Distribution of continuous latent embeddings of an evaluation audio sample before the rounding operation (a) with standard FSQ, and (b) with FSQ-dropout with p=0.75. FSQ-dropout encourages a more uniform distribution, utilizing the full range between -1 and 1.

limiting expressiveness. Even if the encoder did produce a more uniform distribution of continuous values, we would be forced to apply the FSQ rounding operation before decoding, thus rounding away the additional information, since during training FSQ is always enabled. FSQ-dropout addresses this: during training, with probability p, we bypass FSQ's rounding step, feeding the continuous $\tanh(\mathbf{z})$ directly to the upsampler; otherwise, we apply standard FSQ rounding:

$$\tilde{\mathbf{z}} = \begin{cases} \tanh(\mathbf{z}), & \text{with probability } p \\ \frac{\text{round}(N \cdot \tanh(\mathbf{z}))}{N}, & \text{with probability } 1-p \end{cases} \quad (4)$$

where choosing N results in $2N+1$ FSQ quantization levels. This encourages the encoder to produce more informative continuous embeddings across the full $[-1, 1]$ range (Fig. 2(b)) and trains the decoder to accept both discrete and continuous inputs, enabling higher-fidelity continuous reconstruction at inference. We note that a similar technique is proposed in [36], using a combination of FSQ and uniform noise dithering.

Random Mixing: We also introduce *random mixing* as a data augmentation technique. With a probability of 0.5, two randomly selected training samples are mixed (added together) to create a new training sample. This encourages the model to be robust to complex audio scenes with multiple sources. We ablate the effectiveness of this technique in Section 5.

4.2 Decoding Process

CoDiCodec supports two decoding strategies: autoregressive decoding, and a novel *parallel decoding* strategy.

Autoregressive Decoding: Autoregressive decoding is well-suited for interactive applications requiring low latency. In this mode, CoDiCodec generates audio sequentially, chunk by chunk, conditioning the generation of each new chunk on the previously decoded one. For a detailed formalization, we refer the reader to the Music2Latent2 paper [18].

Parallel Decoding: While autoregressive decoding is suitable for interactive applications, it can be inefficient for decoding long sequences, as each chunk must be processed sequentially. We introduce a novel *parallel decoding* strategy that addresses this limitation.

At a high level, we decode adjacent pairs of compressed latents in parallel, and shift the pairs by one at each denoising step to avoid boundary artifacts. More specifically, given a sequence of T sets of summary embeddings, each set encoding information about an audio chunk, we split them into $\lceil T/2 \rceil$ pairs. If T is odd, the last set is paired with a set of zeroed-out summary embeddings. Each pair of summary embeddings is then processed independently. The decoding process involves multiple denoising steps (S). *Step 1*: Each pair of summary embeddings is decoded by the consistency model in parallel, starting from pure noise representations for both the left and right chunks. *Step s* ($1 < s \leq S$): The previously decoded chunks are concatenated, and the *pairs are shifted by one position*. For example, if chunks 0 and 1 were paired in the previous step, chunks 1 and 2 are paired in the current step. Gaussian noise with a decreasing standard deviation $\sigma_{\text{cond},s}$ is added to all chunks. The consistency model then denoises each pair of chunks, conditioned on the corresponding summary embeddings. A linearly decreasing noise schedule ensures that the model gradually refines the decoded audio samples.

This iterative process, with shifting pairs, effectively allows information to propagate across the sequence, mitigating boundary artifacts that would arise from independently decoding fixed pairs. The number of steps, S, controls the trade-off between computational cost and reconstruction quality. While the memory usage of autoregressive decoding is constant regardless of the length of the sequence, for parallel decoding it scales linearly with length (number of chunks), since the model performs multiple decoding steps at the same time.

4.3 Implementation Details

Architecture: CoDiCodec features a scaled-up architecture compared to Music2Latent2 [18], prioritizing transformer blocks over convolutional layers for ease of scalability [37, 38]. The STFT representation uses hop=1024, compared to 512 in Music2Latent, and window=2048. The convolutional patchifiers and de-patchifiers have 5 resolution levels, compared to 7 in Music2Latent2. We use [3, 3, 3, 1] convolutional layers per level, and [64, 128, 256, 512] channels per level. Downsampling/upsampling are performed 3 times, with a factor of 2 along both time and frequency, except for the middle level, where only the frequency axis is downsampled/upsampled by a factor of 4. The encoder, upsampler, and consistency model each have 12 transformer blocks. These blocks have a hidden_dim=512 (compared to 256 in Music2Latent2), head_dim=128, and mlp_mult=4. For each input chunk, the encoder produces $K = 128$ summary embeddings, each with a dimensionality of $d_{lat} = 4$. We can then reshape them to 8 embeddings with 64 channels (resulting in ~11 Hz representations for stereo 44.1 kHz audio). Since they are not a temporally ordered sequence, they can be freely reshaped for different time-dimension vs. channels trade-offs. Noise levels (σ) are encoded using sinusoidal embeddings [33] with 512 channels. Training uses audio samples of 67,072 samples (approximately 1.5 seconds at 44.1 kHz), with STFT spectrograms split into two consecutive 32-frame chunks. We use a batch size of 20 and train for 2 million iterations. We use RAdam [39] with a learning rate of 1×10^{-4}, $\beta_1 = 0.9$, and $\beta_2 = 0.999$.

A cosine learning rate decay is applied, reaching a final learning rate of 0. An Exponential Moving Average (EMA) of the model parameters is maintained with a momentum of 0.9999. For FSQ, we use $N = 5$, resulting in 11 quantization levels per dimension and an implicit codebook size of $11^4 = 14'641$, which is much lower than what modern LLMs [40–42] use. Given the 128 tokens per chunk, this results in a 2.38 kbps rate for stereo 44.1 kHz audio. FSQ-dropout is used with $p = 0.75$, following our ablation results. We use the consistency training framework of [17], with an initial consistency step of $\Delta t_0 = 0.1$ and a final exponent of $e_K = 2$. Random mixing data augmentation is applied with a probability of 0.5. Training is performed on a single A100 GPU and takes ~two weeks. The model has ~150 million parameters.

5. EXPERIMENTS AND RESULTS

Data: We train CoDiCodec on a combination of three datasets: MTG-Jamendo [43] for music (3k hours), the speech (800 hours) and general audio (200 hours) samples from DNS Challenge 4 [44], and M4singer [45] for singing voice (30 hours). We sample the training datasets with weights $[4, 1.5, 4, 1]$, respectively, during training. We choose these weights in order to train CoDiCodec to be robust to speech and general audio, while still focusing on music. Since we are mainly interested in the performance of our model on musical samples, we use MusicCaps [6] as the evaluation dataset. We manually verify that none of the samples in MusicCaps are present in the training sets.

Baselines: For continuous representation baselines, we include: Musika [12], an autoencoder reconstructing magnitude and phase spectrograms; LatMusic [13], an autoencoder designed for latent diffusion models in music accompaniment generation; Moûsai [11], which provides two diffusion autoencoder models (v2 and v3) with differing compression ratios; Music2Latent [17] and Music2Latent2 [18], two consistency-based autoencoders; and the autoencoder used in Stable Audio Open [15, 16]. All these models have compression ratios from 32x to 128x, calculated as waveform values in divided by latent values out. We also include Descript Audio Codec (DAC) [3], a high-fidelity RVQ-based autoencoder producing discrete representations, using both its 2.67 kbit/s and 8 kbit/s configurations.

Metrics: We use: *SI-SDR* (Scale-Invariant Signal-to-Distortion Ratio) [46], which measures the distance between the reconstructed and original waveforms; *ViSQOL* (Virtual Speech Quality Objective Listener) [47–49], which estimates pair-wise perceptual audio quality, providing a MOS-like score; *FAD* (Fréchet Audio Distance) [21], which measures the distance between the distributions of real and generated audio features from a pretrained VGGish [50], assessing overall audio quality; *FAD_clap*, a variant of FAD that uses CLAP [51] features, shown to better correlate with human perception of audio quality [52].

5.1 Ablation Study

We conduct an ablation study to validate the key design choices of CoDiCodec. We train all ablated models for 400k iterations with a batch size of 20, keeping other training parameters and dataset consistent with the full model.

	Continuous		Discrete	
Model	$\text{FAD}_{\text{clap}} \downarrow$	FAD $\downarrow$	$\text{FAD}_{\text{clap}} \downarrow$	FAD $\downarrow$
M2L2	0.0218	0.784	-	-
+ mix aug.	0.0208	0.745	-	-
+ new arch.	0.0178	0.635	-	-
+ 128 lat.	0.0154	0.568	-	-
+ FSQ	-	-	0.0182	0.704
d.o. p=0.25	0.0173	0.628	0.0184	0.725
d.o. p=0.5	0.0169	0.618	0.0191	0.718
d.o. p=0.75	0.0161	0.599	0.0187	0.716

Table 1. Incremental ablation study.

We start by evaluating the same architecture presented in Music2Latent2. We then incrementally add changes to individually evaluate their effect. We first use the random mixing augmentation, then change to our proposed architecture, then use 128 4-dimensional summary embeddings instead of 8 64-dimensional summary embeddings (both having the same total dimensionality and resulting compression ratio), and finally use FSQ-dropout with varying values of the dropout probability p. For each configuration, we report FAD and FAD_{clap} for both continuous embeddings and discrete tokens, when applicable. Table 1 shows how using the random mixing augmentation, changing to our proposed architecture, and re-distributing the same latent space dimensionality from 8 latents with 64 channels to 128 latents with 4 channels, all independently contribute to lower FAD_{clap} and FAD. Introducing FSQ performs slightly worse (as expected, due to quantization), but enables discrete tokens. FSQ-dropout with $p = 0.75$ allows us to both recover a similar discrete tokens performance as the standard FSQ variant, and similar continuous embeddings performance as the fully continuous variant. We thus use this configuration for the remaining experiments.

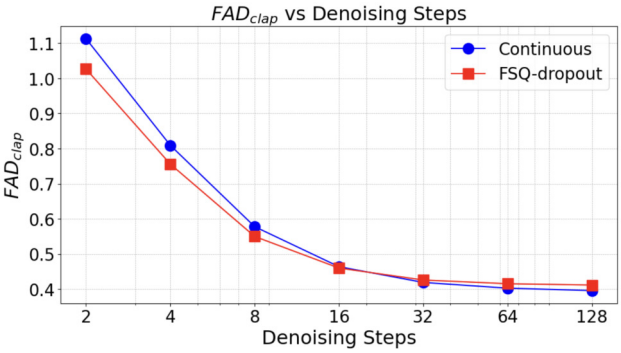


Figure 3. Downstream generative modeling FAD_{clap} with respect to number of denoising steps.

5.2 Downstream Generative Modeling

To assess the impact of the introduced compressed embeddings FSQ-based constraint on downstream generative modeling, we train unconditional generative models using Rectified Flow [53] on continuous latent representations from two configurations:

1. *Continuous*: Embeddings from the "+ 128 lat." model from the ablation study (Section 5.1), which does not use any FSQ, but a simple tanh bottleneck. The distribution of

Model	Stereo	Representation	Compression Ratio	Bitrate	SI-SDR ↑	ViSQOL ↑	FAD_{clap} ↓	FAD ↓
Musika	✗	Continuous	64x	-	-25.81	3.80	0.103	2.308
LatMusic	✗	Continuous	64x	-	-27.32	3.95	0.050	1.630
Moûsai_v2	✓	Continuous	64x	-	-21.44	2.36	0.731	4.687
Moûsai_v3	✓	Continuous	32x	-	-17.47	2.28	0.647	4.473
Music2Latent	✗	Continuous	64x	-	-3.85	3.84	0.036	1.176
Music2Latent2	✓	Continuous	**128x**	-	-2.29	3.91	0.023	0.717
Stable Audio	✓	Continuous	64x	-	**6.04**	**4.08**	0.107	1.017
CoDiCodec (AR)	✓	Continuous	**128x**	-	-0.28	3.95	0.0120	0.390
CoDiCodec (Par., s=3)	✓	Continuous	**128x**	-	-0.08	3.94	0.0114	0.355
CoDiCodec (Par., s=4)	✓	Continuous	**128x**	-	-0.01	3.95	**0.0112**	**0.344**
DAC	✗	Discrete	-	2.67 kbps	2.80	3.87	0.174	3.791
DAC	✗	Discrete	-	8 kbps	**9.48**	**4.21**	0.041	0.966
CoDiCodec (AR)	✓	Discrete	-	**2.38 kbps**	-0.95	3.89	0.0136	0.485
CoDiCodec (Par., s=3)	✓	Discrete	-	**2.38 kbps**	-0.74	3.88	0.0130	0.431
CoDiCodec (Par., s=4)	✓	Discrete	-	**2.38 kbps**	-0.66	3.90	**0.0127**	**0.427**

Table 2. Audio quality and reconstruction metrics.

the latent values follows a gaussian-like distribution, which we scale to have unit standard deviation for the training data.

2. *FSQ-dropout*: Embeddings from the "d.o. p=0.75" model, taken without the FSQ rounding operation. In this case, we first apply an atanh operation to project the FSQ-dropout continuous values from a uniform (Fig. 2(b)) into a comparable gaussian-resembling distribution, and then rescale them to have unit standard deviation.

For each setting, we train a ~100M parameter Rectified Flow DiT [54] for 200k iterations with a batch size of 128, using latents of 10-second samples. We use an internal dataset of 100k single instrument sources as training data. We then generate 1000 samples and evaluate them using FAD_{clap}, varying the number of DiT denoising steps during generation. In Fig. 3 we show that while both configurations converge to a comparable FAD_{clap} with a large number of denoising steps, the model trained on FSQ-dropout embeddings (Setting 2) achieves slightly lower FAD_{clap} when using less than 32 denoising steps. We hypothesize that the implicit regularization provided by FSQ-dropout can be beneficial for latent generative modelling: the decoder appears to be slightly more "robust" to noisy generations of the downstream model. We will further investigate this hypothesis in future work.

Model	Encoding (s)	Decoding (s)
Music2Latent2 (AR)	0.44	4.53
Ours (AR)	0.34	3.22
Ours (Par. s=3)	0.34	2.23
Ours (Par. s=4)	0.34	2.89
Ours (Par. s=5)	0.34	3.51

Table 3. Inference speed comparison (60-second audio).

5.3 Audio Quality and Reconstruction

We evaluate CoDiCodec trained as described in Sec. 4.3. Table 2 presents the audio quality and reconstruction accuracy results. We evaluate both autoregressive (AR) and parallel (Par. using 3 and 4 denoising steps) decoding. We also evaluate both continuous (Cont.) and discrete (Disc.) representations. CoDiCodec significantly outperforms all continuous autoencoder baselines in terms of FAD and FAD_clap. While some baslines achieve higher SI-SDR and ViSQOL, they are explicitly trained with reconstruction losses, while CoDiCodec only uses a generative loss: general audio quality is thus prioritised over reconstruction of the exact same signal, which hurts these pairwise metrics. Crucially, the proposed parallel decoding strategy achieves the best audio quality results, for both continuous and discrete representations. We provide samples here [1] .

5.4 Inference Speed

We measure the encoding and decoding speed by processing a 60-second audio sample on a single RTX 3090 GPU. Table 3 shows that CoDiCodec achieves faster encoding than Music2Latent2, and also substantially faster decoding using the exact same autoregressive decoding strategy. Parallel decoding can further provide lower times if using less than 5 steps. Assuming unlimited memory at our disposal for parallel processing, decoding even longer samples would inevitably widen the gap.

6. CONCLUSION

This paper introduced a novel audio autoencoder producing both continuous embeddings and discrete tokens from a single model, trained end-to-end with a single consistency loss. This is achieved via finite scalar quantization and our proposed FSQ-dropout technique, which allows for expressive continuous latents that perform well for downstream generative modelling. CoDiCodec leverages summary embeddings for high compression and supports both autoregressive and a novel, faster parallel decoding strategy, outperforming existing autoencoders in audio quality metrics. A novel architecture is designed for scalability, focusing on transformer layers. Future work will explore scaling up the model, applying it to diverse audio domains, and investigating its representations for a broader range of MIR tasks, fully exploring the potential of unifying compressed continuous and discrete representations under a single model.

[1] sonycslparis.github.io/codicodec

7. ACKNOWLEDGEMENTS

This work is supported by the EPSRC UKRI Centre for Doctoral Training in Artificial Intelligence and Music (EP/S022694/1) and Sony Computer Science Laboratories Paris.

8. REFERENCES

[1] N. Zeghidour, A. Luebs *et al.*, "SoundStream: An End-to-End Neural Audio Codec," *IEEE ACM Trans. Audio Speech Lang. Process.*, vol. 30, 2022.

[2] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *Transactions on Machine Learning Research*, 2023.

[3] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, "High-fidelity audio compression with improved RVQGAN," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[4] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and controllable music generation," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[5] P. Dhariwal, H. Jun *et al.*, "Jukebox: A generative model for music," *arXiv preprint arXiv:2005.00341*, 2020.

[6] A. Agostinelli, T. I. Denk *et al.*, "MusicLM: Generating Music From Text," Jan. 2023, arXiv:2301.11325 [cs, eess].

[7] I. J. Goodfellow, J. Pouget-Abadie *et al.*, "Generative adversarial nets," in *Advances in Neural Information Processing Systems 27*, Dec. 2014.

[8] J. Sohl-Dickstein, E. A. Weiss *et al.*, "Deep unsupervised learning using nonequilibrium thermodynamics," in *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, ser. JMLR Workshop and Conference Proceedings, vol. 37, 2015.

[9] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *International Conference on Learning Representations*, 2021.

[10] J. Ho, A. Jain *et al.*, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[11] F. Schneider, Z. Jin *et al.*, "Mo\^usai: Text-to-Music Generation with Long-Context Latent Diffusion," Jan. 2023, arXiv:2301.11757 [cs, eess].

[12] M. Pasini and J. Schlüter, "Musika! Fast Infinite Waveform Music Generation," in *Proceedings of the 23rd International Society for Music Information Retrieval Conference, ISMIR 2022, Bengaluru, India, December 4-8, 2022*, 2022.

[13] M. Pasini, M. Grachten *et al.*, "Bass accompaniment generation via latent diffusion," in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024.

[14] Z. Evans, C. Carr, J. Taylor, S. H. Hawley, and J. Pons, "Fast timing-conditioned latent audio diffusion," in *Forty-first International Conference on Machine Learning*, 2024.

[15] Z. Evans, J. D. Parker, C. Carr, Z. Zukowski, J. Taylor, and J. Pons, "Long-form music generation with latent diffusion," in *Proceedings of the 25th International Society for Music Information Retrieval Conference, ISMIR 2024, San Francisco, California, USA and Online, November 10-14, 2024*, 2024.

[16] ——, "Stable audio open," in *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.

[17] M. Pasini, S. Lattner, and G. Fazekas, "Music2latent: Consistency autoencoders for latent audio compression," in *Proceedings of the 25th International Society for Music Information Retrieval Conference, ISMIR 2024, San Francisco, California, USA and Online, November 10-14, 2024*, 2024.

[18] ——, "Music2latent2: Audio compression with summary embeddings and autoregressive decoding," in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.

[19] Y. Song, P. Dhariwal *et al.*, "Consistency Models," May 2023, arXiv:2303.01469 [cs, stat].

[20] Y. Song and P. Dhariwal, "Improved techniques for training consistency models," *arXiv preprint arXiv:2310.14189*, 2023.

[21] K. Kilgour, M. Zuluaga *et al.*, "Fréchet audio distance: A reference-free metric for evaluating music enhancement algorithms," in *20th Annual Conference of the International Speech Communication Association (INTERSPEECH)*, Sep. 2019.

[22] A. van den Oord, O. Vinyals *et al.*, "Neural discrete representation learning," in *Advances in Neural Information Processing Systems 30*, Dec. 2017.

[23] A. Razavi, A. van den Oord *et al.*, "Generating diverse high-fidelity images with VQ-VAE-2," in *Advances in Neural Information Processing Systems 32*, Dec. 2019.

[24] Q. Yu, M. Weber, X. Deng, X. Shen, D. Cremers, and L.-C. Chen, "An image is worth 32 tokens for reconstruction and generation," *arXiv preprint arXiv:2406.07550*, 2024.

[25] S. Luo, Y. Tan, L. Huang, J. Li, and H. Zhao, "Latent consistency models: Synthesizing high-resolution images with few-step inference," *arXiv preprint arXiv:2310.04378*, 2023.

[26] Z. Ye, W. Xue *et al.*, "Comospeech: One-step speech and singing voice synthesis via consistency model," in *Proceedings of the 31st ACM International Conference on Multimedia, MM 2023, Ottawa, ON, Canada, 29 October 2023- 3 November 2023*, 2023.

[27] J. Song, C. Meng *et al.*, "Denoising Diffusion Implicit Models," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021.

[28] F. Mentzer, D. Minnen, E. Agustsson, and M. Tschannen, "Finite scalar quantization: VQ-VAE made simple," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*, 2024.

[29] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.

[30] J. Nistal, S. Lattner *et al.*, "DRUMGAN: synthesis of drum sounds with timbral feature conditioning using generative adversarial networks," in *Proceedings of the 21th International Society for Music Information Retrieval Conference (ISMIR)*, Oct. 2020.

[31] J. Nistal, S. Lattner, and G. Richard, "Comparing representations for audio synthesis using generative adversarial networks," in *28th European Signal Processing Conference (EUSIPCO)*, Jan. 2020.

[32] J. Richter, S. Welker *et al.*, "Speech enhancement and dereverberation with diffusion-based generative models," *IEEE ACM Trans. Audio Speech Lang. Process.*, vol. 31, 2023.

[33] A. Vaswani, N. Shazeer *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems 30*, Dec. 2017.

[34] T. Karras, M. Aittala *et al.*, "Elucidating the Design Space of Diffusion-Based Generative Models," Oct. 2022, arXiv:2206.00364 [cs, stat].

[35] Z. Geng, A. Pokle, W. Luo, J. Lin, and J. Z. Kolter, "Consistency models made easy," in *The Thirteenth International Conference on Learning Representations*, 2025.

[36] J. D. Parker, A. Smirnov, J. Pons, C. Carr, Z. Zukowski, Z. Evans, and X. Liu, "Scaling transformers for low-bitrate high-quality speech coding," *arXiv preprint arXiv:2411.19842*, 2024.

[37] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[38] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[39] L. Liu, H. Jiang *et al.*, "On the variance of the adaptive learning rate and beyond," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020.

[40] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[41] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[42] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[43] D. Bogdanov, M. Won *et al.*, "The mtg-jamendo dataset for automatic music tagging," in *Machine Learning for Music Discovery Workshop, International Conference on Machine Learning (ICML 2019)*, Long Beach, CA, United States, 2019.

[44] H. Dubey, V. Gopal *et al.*, "Icassp 2022 deep noise suppression challenge," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2022, Virtual and Singapore, 23-27 May 2022*, 2022.

[45] L. Zhang, R. Li, S. Wang, L. Deng, J. Liu, Y. Ren, J. He, R. Huang, J. Zhu, X. Chen, and Z. Zhao, "M4singer: A multi-style, multi-singer and musical score provided mandarin singing corpus," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 6914–6926.

[46] J. L. Roux, S. Wisdom *et al.*, "SDR - half-baked or well done?" in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2019, Brighton, United Kingdom, May 12-17, 2019*, 2019.

[47] A. Hines, J. Skoglund *et al.*, "Visqol: an objective speech quality model," *EURASIP J. Audio Speech Music. Process.*, vol. 2015, 2015.

[48] C. Sloan, N. Harte *et al.*, "Objective assessment of perceptual audio quality using visqolaudio," *IEEE Trans. Broadcast.*, vol. 63, no. 4, 2017.

[49] M. Chinen, F. S. C. Lim *et al.*, "Visqol v3: An open source production ready objective speech and audio metric," in *Twelfth International Conference on Quality of Multimedia Experience, QoMEX 2020, Athlone, Ireland, May 26-28, 2020*, 2020.

[50] S. Hershey, S. Chaudhuri, D. P. W. Ellis, J. F. Gemmeke, A. Jansen, R. C. Moore, M. Plakal, D. Platt, R. A. Saurous, B. Seybold, M. Slaney, R. J. Weiss, and K. W. Wilson, "CNN architectures for large-scale audio classification," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2017, New Orleans, LA, USA, March 5-9, 2017*. IEEE, 2017, pp. 131–135.

[51] Y. Wu, K. Chen *et al.*, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," in *IEEE International Conference on Acoustics, Speech and Signal Processing ICASSP 2023, Rhodes Island, Greece, June 4-10, 2023*, 2023.

[52] M. Tailleur, J. Lee *et al.*, "Correlation of fr\'echet audio distance with human perception of environmental audio is embedding dependant," *arXiv preprint arXiv:2403.17508*, 2024.

[53] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*, 2023.

[54] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, 2023.